LibIHT: A Hardware-Based Approach to Efficient and Evasion-Resistant Dynamic Binary Analysis

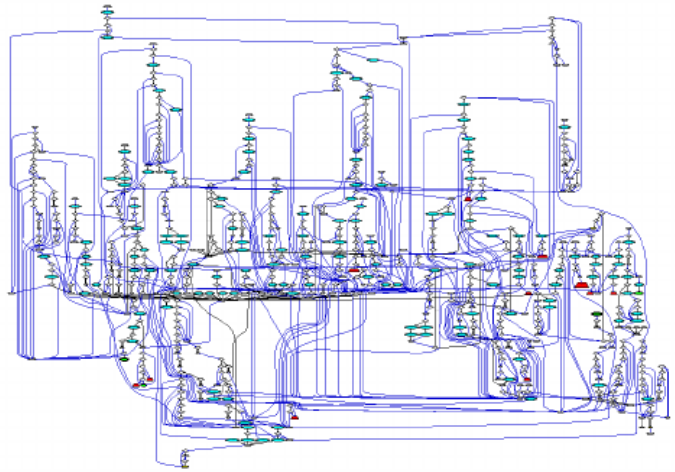
Changyu Zhao, Yohan Beugin, Jean-Charles Noirot Ferrand, Quinn Burke, Guancheng Li, Patrick McDaniel

Semiconductor Research Corporation

ACM CCS SURE 2025

Problem & Motivation

- **Tracing** is the art of understanding and reconstructing the execution behavior by running the program.
- Dynamic Binary Instrumentation (DBI) Tools are detailed but **slow** and **detectable**.
- Anti-analysis checks often catch DBI, we need **stealth** + **speed**.

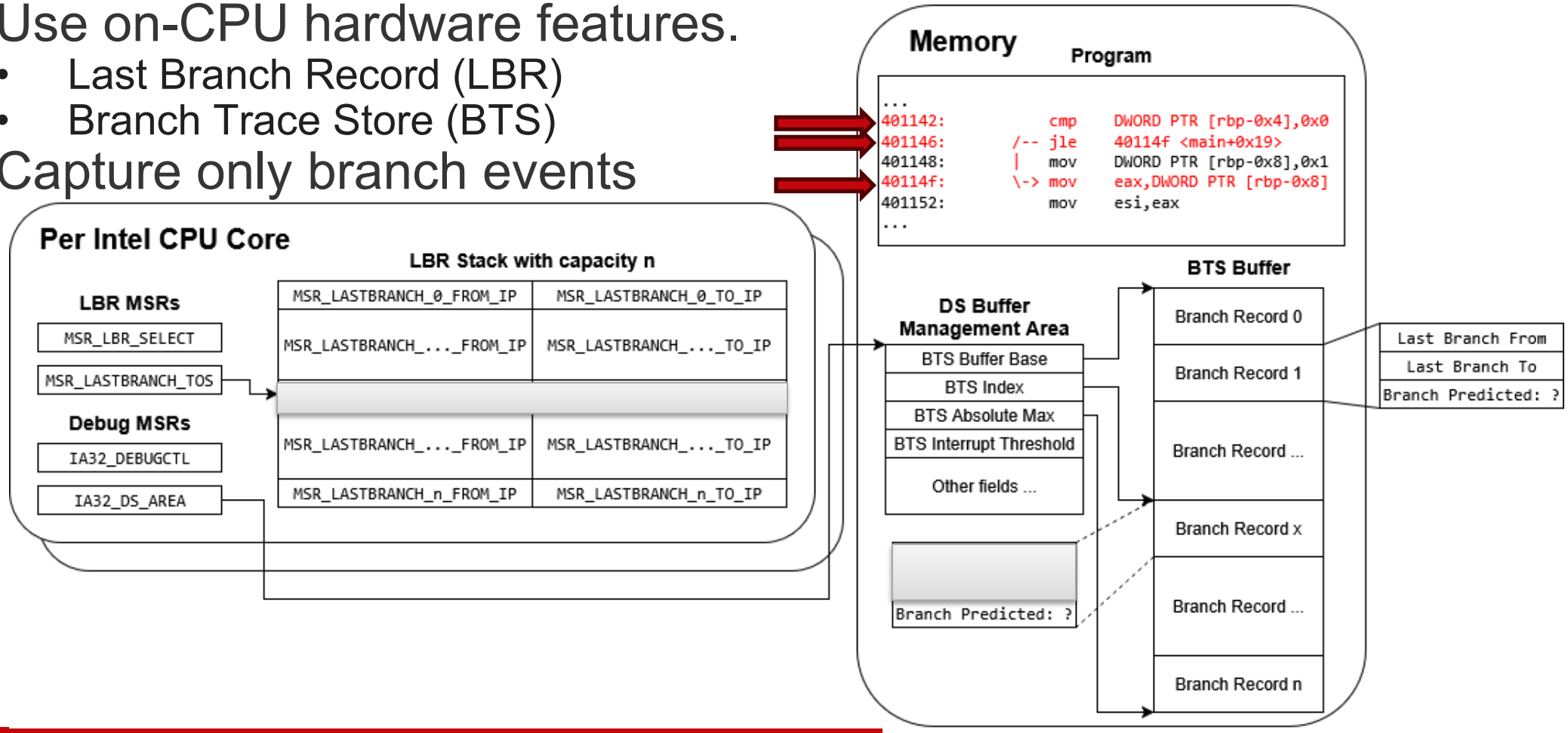


Program	Native	Intel Pin	DynamoRIO
ls	7 ms	829 ms (118×)	94 ms (13×)
dd	14 ms	625 ms (45×)	68 ms (5×)
echo	4 ms	508 ms (127×)	54 ms (14×)
sort	9 ms	600 ms (67×)	63 ms (7×)
wc	8 ms	559 ms (70×)	56 ms (7×)
cat	7 ms	497 ms (71×)	55 ms (8x)

Instrumentation overhead of Intel Pin and DynamoRIO **without** any analysis logic.

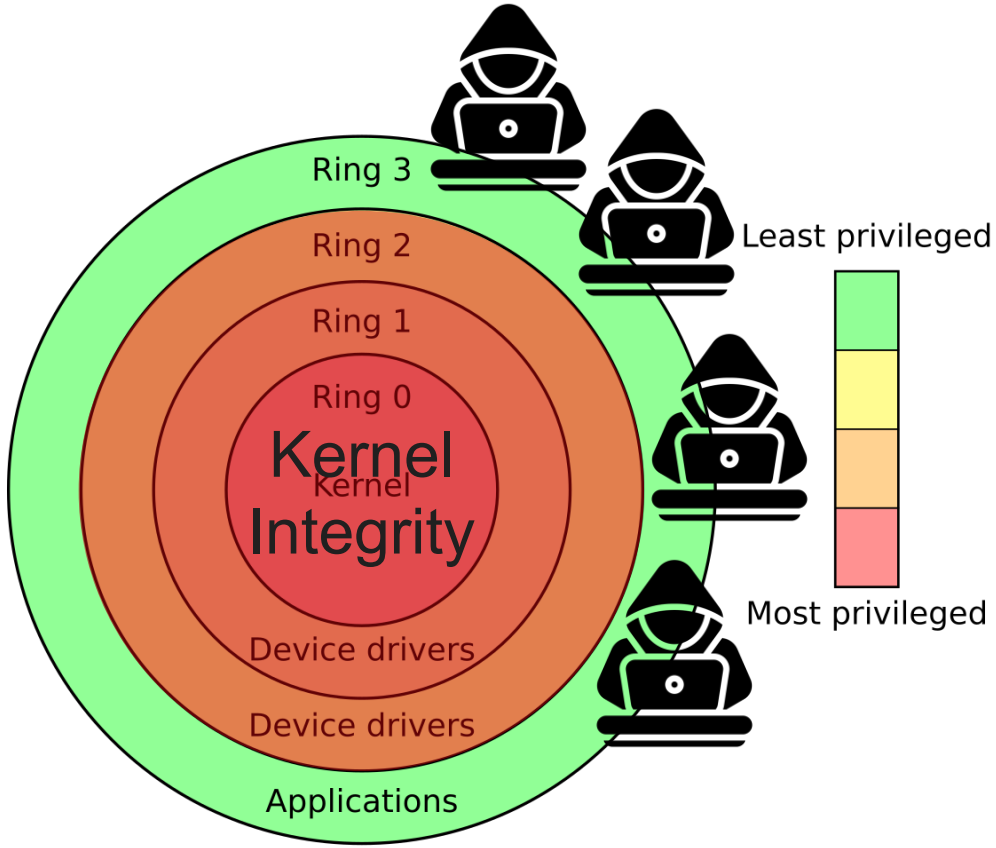
Key Idea: let the CPU tell us where control flows

- Use on-CPU hardware features.
  - Last Branch Record (LBR)
  - Branch Trace Store (BTS)
- Capture only branch events



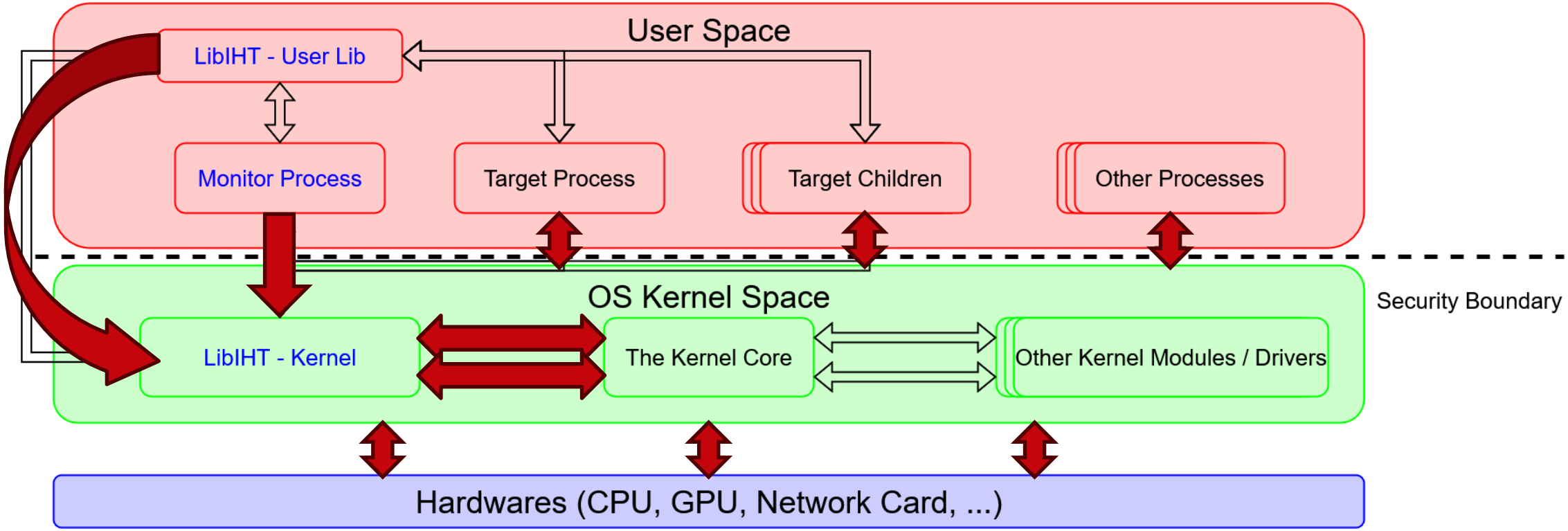
Threat Model

- Adversary in user space; kernel (ring-0) integrity holds.
- Put tracing logic in kernel to resist tampering.
- Goal: preserve behavior under anti-instrumentation/evasion



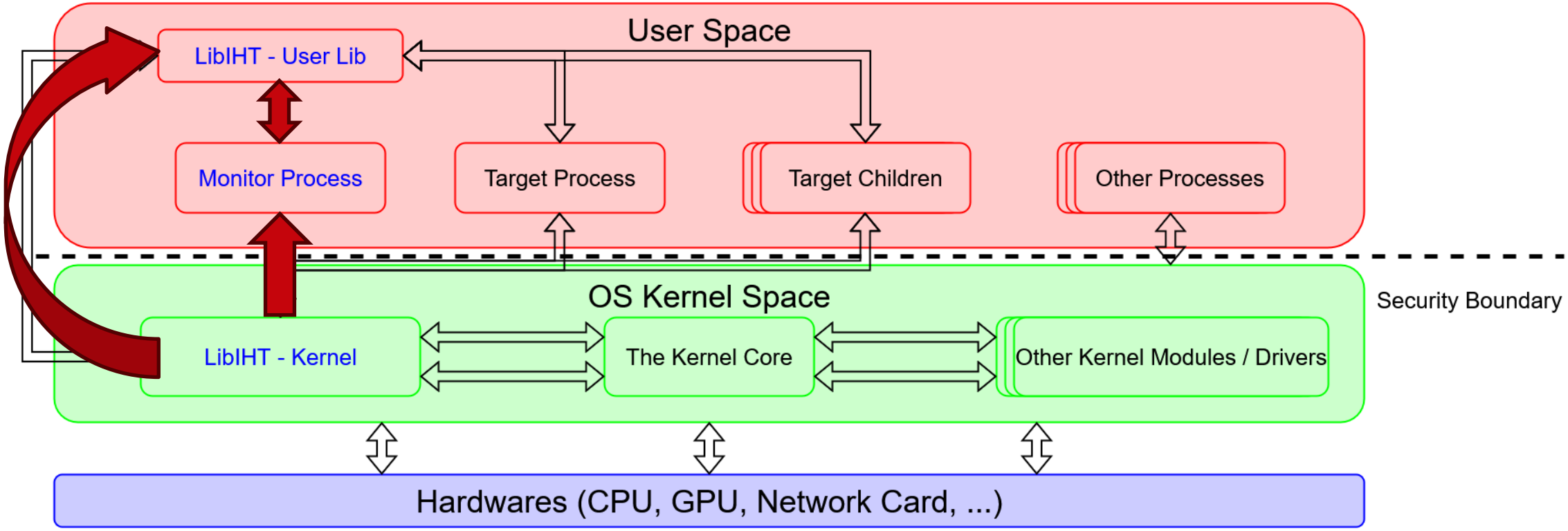
System Overview (Data Collection)

- Issue ioctl to kernel to setup tracing session for target PID(s).
- Kernel programs LBR/BTS and keeps per-target state/buffers.



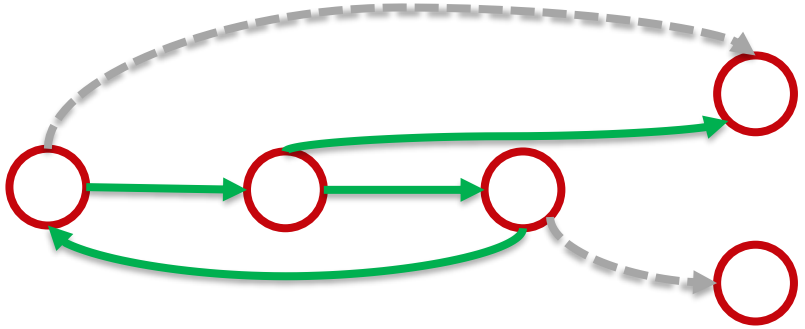
System Overview (CFG Reconstruction)

- Read branch records from kernel buffers.
- Symbolize addresses and map to basic blocks/static CFG.



LibIHT Accuracy

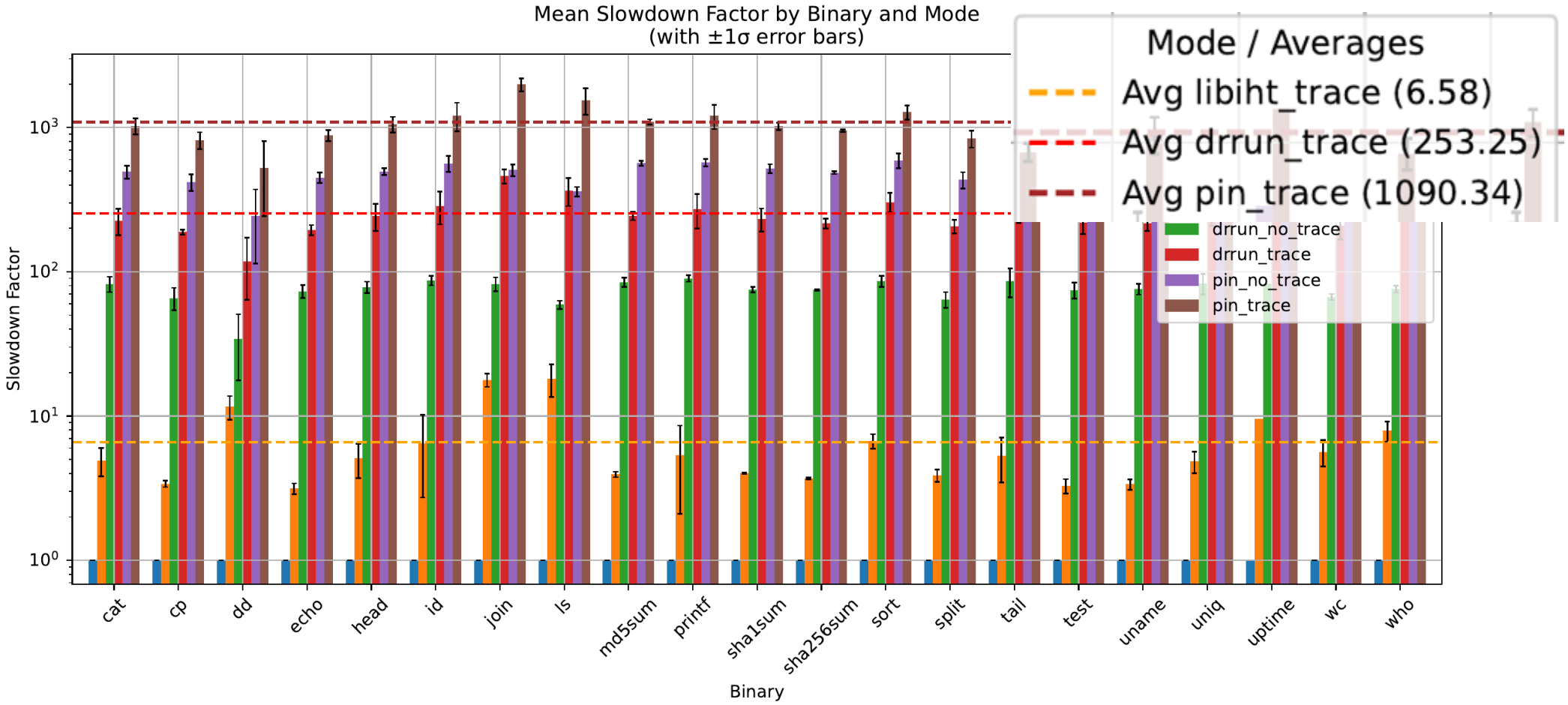
- Project branch trace onto static CFG to isolate executed path.
- Kernel-boundary causing "inaccurate" results.



Metric	LibIHT	Intel Pin	DynamoRIO
Jaccard Index	98.36 ± 0.14%	100%	100%
Normalized GED	0.0231	≈ 0	≈ 0
Block Coverage	99.92 ± 0.04%	100%	100%
Edge Coverage	99.48 ± 0.15%	100%	100%

Control-Flow Graph Reconstruction Metrics on Benign Benchmarks

LibIHT Performance



LibIHT Adversarial Resilience

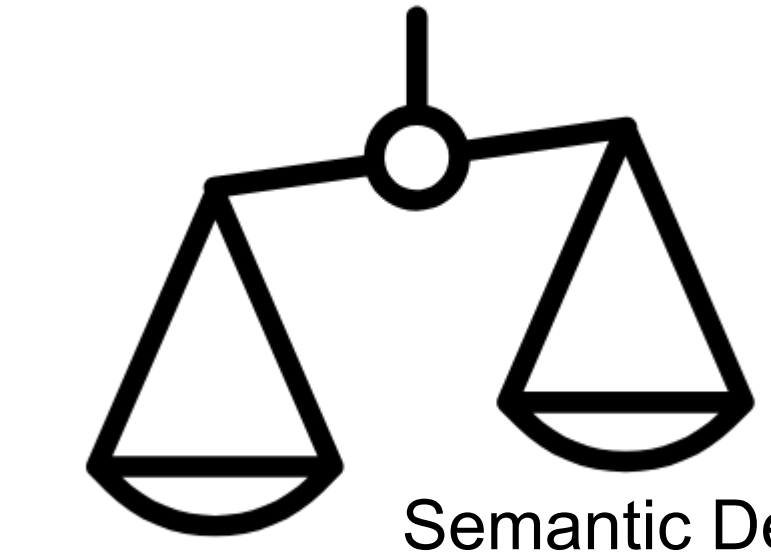
- LibIHT **undetected** across checks; Pin/DRIO detected by several.
- Accuracy under evasion: LibIHT ≥98.7% block coverage
- Overhead under evasion: LibIHT ≤ 14.5× worst-case; DBI often 100×~600×

Sample	Intel Pin	DynamoRIO	LibIHT
fl_fsbase.c	✗	✓	✗
fl_ripsyscall.c	✗	✓	✗
ca_nx.c	✗	✓	✗
ca_smc.c	✓	✗	✗
ca_vmleave.c	✓	✗	✗
ea_envvar.c	✓	✓	✗
ea_mapname.c	✓	✓	✗
ea_pageperm.c	✓	✓	✗
ro_jitbr.c	✓	✓	✗
ro_jitlib.c	✗	✗	✗

	Block Coverage (%)			Slowdown Factor (×)		
Sample	Intel Pin	DynamoRIO	LibIHT	Intel Pin	DynamoRIO	LibIHT
fl_fsbase.c	100%	N/A	99.1%	674.09	N/A	1.67
fl_ripsyscall.c	100%	N/A	99.3%	518.58	N/A	1.26
ca_nx.c	100%	N/A	98.7%	677.57	N/A	1.86
ca_smc.c	N/A	100%	98.8%	N/A	170.13	1.84
ca_vmleave.c	N/A	100%	99.9%	N/A	1352.34	12.76
ea_envvar.c	N/A	N/A	99.9%	N/A	N/A	3.28
ea_mapname.c	N/A	N/A	99.9%	N/A	N/A	5.32
ea_pageperm.c	N/A	N/A	99.9%	N/A	N/A	5.41
ro_jitbr.c	N/A	N/A	99.7%	N/A	N/A	1.51
ro_jitlib.c	100%	100%	99.3%	678.28	158.03	14.52

Limitations

- Branch addresses only → less semantic detail than DBI.
- Potential micro-arch attack surface (future evasion).
- Pair with selective instrumentation for deep analysis.



Takeaways

- We present LibIHT, a **hardware-assisted** tracing framework.
- It achieves **near-perfect** reconstruction of executed CFGs from hardware branch traces, remaining both **fast** and **stealthy** across benchmarks.
- The system works on unmodified binaries, add selective instrumentation only when more data semantics are needed.

LibIHT

Intel Hardware Trace Library

Contact: changyuz@stanford.edu

Website: blog.thomasonzhao.cn

THANK YOU!

Q & A


WISCONSIN
UNIVERSITY OF WISCONSIN–MADISON

LibIHT Demo: GDB